

PyTorch autograd 模块设计（分析）报告

范潇

2025 年 5 月 30 日

1 引言

1.1 软件描述

PyTorch 的 `autograd` 模块是其自动微分系统的核心，基于反向模式自动微分（Reverse Mode AD）实现。它通过动态计算图机制，在每次对带有 `requires_grad=True` 的张量执行操作时自动构建图结构。每个张量记录其生成方式，并在调用 `backward()` 方法时，从输出开始按链式法则向后传播梯度，最终将梯度累积到各个叶子张量的 `.grad` 属性中。这一机制无需用户手动求导，使得深度学习模型的训练过程高度自动化且高效。

1.2 人员分工

本模块的分析全部由范潇完成。

2 软件系统设计（分析）

2.1 软件系统架构

PyTorch `autograd` 模块的系统架构围绕动态计算图机制设计，由张量对象系统、操作函数节点（Function）、计算图管理器、梯度调度引擎和上下文存储系统等组成。每个张量记录其生成历史，操作被封装为 `Function` 节点并形成有向无环图。前向计算时构建图结构，反向传播时由引擎驱动节点逆序调用其 `backward` 方法计算梯度。尽管动态计算图本身是一种执行策略而非模块，但它是整个 `autograd` 架构设计的核心基础。

2.2 Autograd 模块接口说明

2.2.1 基础自动微分接口

这些接口用于构建和执行计算图，支持反向传播和梯度计算：

- `torch.autograd.backward(tensors, grad_tensors=None, ...)`: 对给定张量执行反向传播，计算其对叶子节点的梯度。
- `torch.autograd.grad(outputs, inputs, grad_outputs=None, ...)`: 计算输出对输入的梯度，不会影响计算图。
- `torch.Tensor.backward(gradient=None, ...)`: 张量对象的方法，触发反向传播。
- `torch.Tensor.grad`: 存储张量的梯度值。
- `torch.Tensor.requires_grad`: 是否需要计算梯度。
- `torch.Tensor.retain_grad()`: 在非叶子张量上保留梯度。
- `torch.Tensor.register_hook(hook)`: 注册钩子函数，用于调试或修改梯度。

2.2.2 前向模式自动微分 (Forward-mode AD)

这些接口用于在前向传播过程中同步计算输入变化对输出的影响，适合小输入、大输出场景：

- `torch.autograd.forward_ad.dual_level()`: 定义前向自动微分的作用域。
- `torch.autograd.forward_ad.make_dual(primal, tangent)`: 创建双重张量 (dual tensor)。
- `torch.autograd.forward_ad.unpack_dual(dual_tensor)`: 解包双重张量。
- `torch.autograd.forward_ad.enter_dual_level()`, `exit_dual_level()`: 手动进入或退出前向 AD 层。
- `torch.autograd.forward_ad.UnpackedDualTensor`: 解包后返回的命名元组。

2.2.3 函数式自动微分接口

这些接口以函数式编程方式进行自动微分，适合高阶微分（如雅可比矩阵、海森矩阵）计算场景：

- `torch.autograd.functional.jacobian(func, inputs, ...)`: 计算雅可比矩阵。
- `torch.autograd.functional.hessian(func, inputs, ...)`: 计算海森矩阵。
- `torch.autograd.functional.jvp(func, inputs, v, ...)`: 计算雅可比向量积 (JVP)。
- `torch.autograd.functional.vjp(func, inputs, v, ...)`: 计算向量雅可比积 (VJP)。
- `torch.autograd.functional.hvp(func, inputs, v, ...)`: 计算海森向量积 (HVP)。
- `torch.autograd.functional.vhp(func, inputs, v, ...)`: 计算向量海森积 (VHP)。

2.2.4 梯度计算控制接口

这些接口用于控制梯度的计算行为，适用于推理阶段或特定的训练策略，帮助节省内存和加速推理过程：

- `torch.no_grad()`: 禁用梯度计算，用于推理阶段。
- `torch.enable_grad()`: 启用梯度计算。
- `torch.set_grad_enabled(mode)`: 根据布尔值动态启用或禁用梯度计算。
- `torch.inference_mode()`: 禁用梯度与版本计数，提升推理性能。

2.2.5 自定义自动微分函数

这些接口用于实现自定义操作的前向与反向传播逻辑，适用于扩展 PyTorch 的自动微分能力：

- `torch.autograd.Function`: 定义自定义操作，需要实现 `forward` 和 `backward` 方法。
- `ctx.save_for_backward(*tensors)`: 在前向中保存张量，供反向使用。
- `ctx.saved_tensors`: 访问保存的张量。
- `ctx.mark_non_differentiable(*tensors)`: 标记张量为不可微分。
- `ctx.set_materialize_grads(value)`: 控制是否为未使用输入生成梯度。

2.2.6 梯度检查工具

这些工具用于通过数值近似检查梯度实现的正确性，常用于调试和验证自定义算子：

- `torch.autograd.gradcheck(func, inputs, ...)`: 数值近似检查一阶梯度。
- `torch.autograd.gradgradcheck(func, inputs, ...)`: 检查二阶梯度的正确性。

2.2.7 其他实用工具

这些接口用于调试、异常检测和性能分析，帮助开发者优化模型的稳定性和执行效率：

- `torch.autograd.detect_anomaly()`: 检测反向传播中的异常。
- `torch.autograd.set_detect_anomaly(mode)`: 启用或禁用异常检测。
- `torch.autograd.profiler.profile()`: 性能分析器，分析计算开销。
- `torch.autograd.profiler.record_function(name)`: 手动标记分析代码块。

2.3 软件模块依赖关系

Autograd 模块中各文件的依赖关系如下：

- `variable.py`: 仅用于兼容早期版本的 `Variable`，本质上是 `torch.Tensor` 的封装，依赖 `torch` 核心模块，无直接逻辑依赖 `function.py`。
- `function.py`: 定义前向/反向传播的 `Function` 节点，依赖 `graph.py` 来管理计算图中的依赖关系（Edges）。
- `graph.py`: 定义计算图的基本结构，包括 `Edge`、`GraphTask` 等，为 `function.py` 和后端调度器（Engine）提供支持。
- `forward_ad.py`: 实现前向模式自动微分，轻度依赖 `variable.py` 中的张量包装机制。
- `grad_mode.py`: 管理全局的梯度计算开关（如 `no_grad`、`enable_grad`），影响计算图的构建行为。
- `anomaly_mode.py`: 用于异常检测，封装了异常追踪逻辑，依赖 `grad_mode.py` 进行状态控制。
- `functional.py`: 提供高阶自动微分接口（如 `Jacobian`、`Hessian` 计算），内部调用 `function.py` 和 `grad`。
- `gradcheck.py`: 用于数值梯度验证，依赖 `functional.py` 提供的高阶微分接口。
- `profiler.py`、`profiler_util.py`、`profiler_legacy.py`: 性能分析模块，hook 到 Autograd Engine 调度过程，收集执行时间和内存信息。

依赖关系可以概括为：

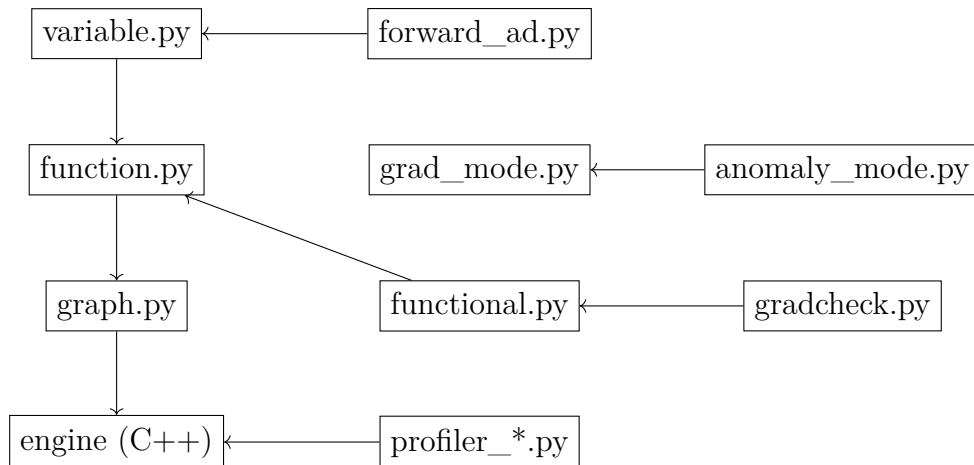


图 1: Autograd 模块内部文件依赖关系图

3 软件详细设计（分析）

3.1 Autograd 模块类设计

Autograd 模块围绕自动微分提供了三大类核心功能，分别是：

- 传统的反向模式自动微分（Backward-Mode AD），以动态计算图（Dynamic Computation Graph）为基础，支持标准的梯度反向传播；
- 正向模式自动微分（Forward-Mode AD），通过 Dual Tensor 机制高效传播输入扰动方向的微分信息，适用于小输入大输出场景；
- 无状态（Functional）自动微分接口，提供向量-雅可比乘积（VJP）、雅可比-向量乘积（JVP）、雅可比矩阵（Jacobian）、海森矩阵（Hessian）等高级微分能力，支持更灵活和高阶的求导需求。

此外，Autograd 模块还包含若干辅助功能模块，如：

- 梯度计算模式管理（如 `no_grad`, `enable_grad`）；
- 异常检测与调试支持（如 `detect_anomaly`）；
- 自动微分性能分析工具（如 `profiler`）。

这些功能虽然对完整的系统实现具有重要意义，但并非自动微分核心机制的直接组成部分，因此在后续分析中将不予展开，以突出主题，控制篇幅。

因此，后文将按照上述三类核心功能，对 Autograd 模块中涉及的主要类设计分别进行归纳与分析，重点揭示各类设计的职责划分、核心机制与相互依赖关系，以便系统性理解 PyTorch 自动微分引擎的内部实现逻辑。

3.1.1 自动微分功能分类与类设计分析说明

`graph.py`

- **Node**: 抽象基类，定义了计算图中每个节点的基本接口。每个反向传播节点（如 `CloneBackward0`）都继承自 `Node`，必须实现 `name`、`next_functions`、`metadata` 等方法，支持注册前向和后向 hook。
- **GradientEdge**: 封装了计算图中张量到其生成节点（Node）的连接关系，包括 `node` 和 `output_nr` 两个属性。
- **saved_tensors_hooks**: 上下文管理器，用于在保存反向传播中间变量时自定义保存与恢复策略（如将中间变量保存到 CPU 以节省 GPU 显存）。

`function.py`

- **FunctionCtx**: 自定义 `Function` 的上下文类，负责保存 `forward` 过程中需要在 `backward` 中使用的张量信息，支持 `save_for_backward`、`mark_dirty` 等方法。

- **FunctionMeta:** 元类, 用于在定义自定义 Function 时自动创建对应的反向传播类 (BackwardCFunction)。
- **Function:** 自定义算子基类, 用户需继承该类并实现静态方法 `forward` 和 `backward`, 支持扩展前向模式 (jvp) 和向量化规则 (vmap)。
- **BackwardCFunction:** C++ 后端定义的基类, 连接 Python 端 Function 和 C++ 调度器 (Engine), 负责在反向传播时调用用户定义的 `backward` 或 `vjp`。
- **once_differentiable:** 装饰器, 用于标记 `backward` 函数只支持一次求导, 不支持二阶梯度。

variable.py

- **VariableMeta:** 用于兼容早期版本, 定义 Variable 和 Tensor 类型一致的 `isinstance` 行为。
- **Variable:** 旧版 Variable 兼容层, 实际上继承自 `torch.Tensor` 内部的 `_LegacyVariableBase`, 并绑定了底层反向传播执行引擎 `ImperativeEngine`。

总结 上述设计共同构成了 PyTorch Autograd 模块的自动微分机制, 核心流程包括:

- 构建计算图: 每个 Tensor 通过其 `grad_fn` 指向生成该 Tensor 的 Function 节点。
- 记录中间状态: 通过 `save_for_backward` 机制保存必要变量。
- 执行反向传播: 由 BackwardCFunction 统一调度, 按依赖关系逐步执行 `backward`, 计算各 Tensor 的梯度。
- 支持扩展: 允许用户自定义新的 Function, 插入到整体计算图中, 支持更复杂的微分需求。

3.1.2 Forward-Mode 自动微分 (Forward AD) 类设计分析

forward_ad.py

- **dual_level:** 上下文管理器类。通过 `enter_dual_level` 和 `exit_dual_level` 接口管理前向 AD 的有效计算范围, 确保 Forward AD 在特定作用域内有效。
- **make_dual:** 函数接口。将普通 Tensor 和对应的切向量 (Tangent) 绑定成一个 Dual Tensor, 作为正向微分的基本输入对象。
- **unpack_dual:** 函数接口。从 Dual Tensor 中提取出原始 Tensor (Primal) 和切向量 (Tangent), 用于正向模式微分结果读取。
- **UnpackedDualTensor:** 简单的数据结构 (NamedTuple), 存储 Primal 和 Tangent 两个 Tensor, 作为 `unpack_dual` 返回值。
- **_set_fwd_grad_enabled** (内部工具类): 用于控制 Forward Grad 功能启用或关闭, 仅用于 PyTorch 内部或高级用例。

总结 在 PyTorch Autograd 模块中, Forward-Mode AD 相关功能采用了清晰分层的设计思路:

- 通过上下文管理 (`dual_level`) 明确 Forward AD 生效范围;
- 通过 Tensor 扩展机制 (`make_dual/unpack_dual`) 封装切向量信息;

3.1.3 无状态自动微分 (Functional API) 类设计分析

`functional.py`

- **vjp**: 向量-雅可比乘积 (Vector-Jacobian Product) 接口。给定函数 `func` 和输入 `inputs`, 计算雅可比矩阵与向量 `v` 的乘积, 基于反向模式自动微分 (Backward-Mode AD) 实现。
- **jvp**: 雅可比-向量乘积 (Jacobian-Vector Product) 接口。给定函数 `func` 和输入 `inputs`, 计算输入方向上的微分, 主要基于反向两次求导 (Backward-Backward Trick) 实现, 也支持调用 Forward-AD。
- **jacobian**: 计算函数的完整 Jacobian 矩阵, 内部可以选择正向模式 (Forward-Mode) 或反向模式 (Reverse-Mode) 策略。
- **hessian**: 计算二阶导数矩阵 (Hessian Matrix), 通过两次应用 Jacobian 接口实现 (即 Jacobian of Jacobian)。
- **vhp**: 向量-海森乘积 (Vector-Hessian Product) 接口, 高效地计算 Hessian 矩阵与向量 `v` 的乘积。
- **hvp**: 海森-向量乘积 (Hessian-Vector Product) 接口, 与 `vhp` 功能类似, 但采用不同的求导策略。

总结 无状态 (Functional) API 提供了比传统 Tensor API 更灵活、更底层的接口:

- 明确区分了输入、输出和切向量, 避免状态副作用;
- 支持 Forward 和 Reverse 两种自动微分策略, 适应不同任务 (如大输入小输出, 或小输入大输出);
- 能高效构建更复杂的求导算子, 如高阶导数 (Hessian)、向量微分产品 (VJP/JVP) 等;
- 通过 `dual_level` 与 `make_dual` 机制, 实现 Forward-Mode AD 与普通反向传播的无缝切换。

3.2 Autograd 模块算法设计

限于篇幅原因, 我们在本节中介绍 Autograd 模块中最重要的两个功能: 正向自动微分以及反向自动微分背后的算法原理。

3.2.1 Forward AD

正向自动微分（Forward Automatic Differentiation, 简称 Forward AD）是一种基于链式法则，在计算过程中传播方向导数的方法。对于函数

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

给定输入点 $x \in \mathbb{R}^n$ 和方向 $\dot{x} \in \mathbb{R}^n$ ，Forward AD 计算方向导数：

$$\dot{y} = J_f(x) \cdot \dot{x},$$

其中 $J_f(x)$ 是在 x 处的雅可比矩阵， $\dot{y} \in \mathbb{R}^m$ 是函数值在方向 $\dot{x}$ 上的变化率。

双数（Dual Numbers）模型 在 Forward AD 中，我们希望在执行函数计算的同时，自动追踪每个变量的导数信息。为此，引入了 **双数（dual number）** 的概念：

$$x^* = x + \varepsilon \dot{x},$$

其中， x 表示变量的数值部分， $\dot{x}$ 表示其导数， ε 是一个满足 $\varepsilon^2 = 0$ 且 $\varepsilon \neq 0$ 的抽象符号。

这个 ε 并非通常意义上的无穷小，而是定义在双数域中的一个特殊符号，其关键性质在于其平方为零。这一代数构造的核心目的是实现链式法则的自然传播。例如，考虑两个双数相乘：

$$(x + \varepsilon \dot{x})(y + \varepsilon \dot{y}) = xy + \varepsilon(x\dot{y} + y\dot{x}) + \varepsilon^2 \dot{x}\dot{y}.$$

若设 $\varepsilon^2 = 0$ ，则有：

$$(xy)^* = xy + \varepsilon(x\dot{y} + y\dot{x}),$$

恰好对应了乘法的求导规则。因此，借助双数结构，我们可以在进行普通数值计算的同时，通过 ε 所携带的“导数项”自动完成导数传播。

更一般地，运算符对双数的扩展可以形式化地写作：

$$\begin{aligned}(x + y)^* &= x + y + \varepsilon(\dot{x} + \dot{y}), \\ (x \cdot y)^* &= xy + \varepsilon(x\dot{y} + y\dot{x}), \\ \sin(x)^* &= \sin(x) + \varepsilon \cos(x)\dot{x}, \quad \text{等等}.\end{aligned}$$

计算图中的传播过程 设复合函数 $y = f_k \circ f_{k-1} \circ \cdots \circ f_1(x)$ ，Forward AD 按照链式法则按复合函数表达式由内向外传播导数：

$$\dot{y} = \frac{\partial f_k}{\partial z_{k-1}} \cdot \frac{\partial f_{k-1}}{\partial z_{k-2}} \cdots \frac{\partial f_1}{\partial x} \cdot \dot{x},$$

其中 $z_i = f_i(z_{i-1})$ ，每个中间变量 z_i 都附带其切向量 $\dot{z}_i$ 。

Forward AD 的核心算法伪代码

Algorithm 1: Forward AD: 正向模式自动微分

Input: 函数计算图 $f_1, f_2, \dots, f_k$, 输入变量值 x , 输入方向导数 $\dot{x}$

Output: 函数值 y , 方向导数 $\dot{y}$

```

1 初始化:  $z_0 \leftarrow x, \dot{z}_0 \leftarrow \dot{x}$ ;
2 for  $i \leftarrow 1$  to  $k$  do
3    $z_i \leftarrow f_i(z_{i-1})$ ;
4    $\dot{z}_i \leftarrow J_{f_i}(z_{i-1}) \cdot \dot{z}_{i-1}$ ;           // 传播方向导数
5  $y \leftarrow z_k, \dot{y} \leftarrow \dot{z}_k$ ;
6 return  $(y, \dot{y})$ 

```

总结 Forward AD 通过在每个变量中嵌入切向量（使用双数结构），在函数执行时同步计算方向导数，避免手动推导复杂表达式。其在高阶导数、Jacobian 向量积、优化器实现等场景中具有重要作用。PyTorch 利用 dual tensor 结构和上下文管理器封装了 Forward AD 的实现，使用户可以在保留张量接口的同时，精确控制方向导数的传播。

3.2.2 Backward AD

反向自动微分（Reverse Automatic Differentiation, 简称 Backward AD）是一种自右向左传播梯度的自动微分方法，特别适用于标量输出函数的高效梯度计算。对于函数

$$f : \mathbb{R}^n \rightarrow \mathbb{R},$$

给定输入 $x \in \mathbb{R}^n$, Backward AD 旨在计算梯度向量:

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right].$$

计算图与反向传播 在 Backward AD 中，首先通过前向执行建立计算图，图中每个节点表示一个中间变量或运算操作，每条边表示依赖关系。随后，从输出节点出发，应用链式法则将梯度从后向前传播。

设复合函数 $y = f_k \circ f_{k-1} \circ \dots \circ f_1(x)$, 其中每个中间变量 $z_i = f_i(z_{i-1})$, 则有:

$$\frac{\partial y}{\partial x} = \frac{\partial z_k}{\partial z_{k-1}} \cdot \frac{\partial z_{k-1}}{\partial z_{k-2}} \dots \frac{\partial z_1}{\partial x}.$$

我们从 $\frac{\partial y}{\partial z_k} = 1$ 开始，逐层计算:

$$\frac{\partial y}{\partial z_{k-1}} = \frac{\partial z_k}{\partial z_{k-1}}^\top \cdot \frac{\partial y}{\partial z_k}.$$

Backward AD 的核心算法伪代码

Algorithm 2: Backward AD: 反向模式自动微分

Input: 函数计算图 $f_1, f_2, \dots, f_k$, 输入变量 x

Output: 输出值 y , 输入梯度 $\nabla y = \frac{\partial y}{\partial x}$

```
1 前向计算: 构建计算图并记录中间变量;
2  $z_0 \leftarrow x$ ;
3 for  $i \leftarrow 1$  to  $k$  do
4    $z_i \leftarrow f_i(z_{i-1})$ ; // 前向传播
5   在图中记录  $\frac{\partial z_i}{\partial z_{i-1}}$ 
6  $y \leftarrow z_k$ , 初始化反向梯度:  $\bar{z}_k \leftarrow 1$ ; // 输出对自身的导数为 1
7 for  $i \leftarrow k$  to 1 do
8    $\bar{z}_{i-1} \leftarrow \bar{z}_i \cdot \left( \frac{\partial z_i}{\partial z_{i-1}} \right)$ ; // 反向传播梯度
9 return  $(y, \bar{z}_0)$ 
```

其中, 符号 $\bar{z}_i = \frac{\partial y}{\partial z_i}$ 表示反向传播过程中每个中间变量对输出的敏感度。

PyTorch 中的实现机制 PyTorch 利用动态图机制构建反向传播图。在执行每个操作时, ‘autograd’ 会记录该操作及其输入输出, 并为其绑定一个特定的 `Function` 对象, 该对象封装了该操作的后向求导规则。调用 `backward()` 后, PyTorch 会自顶向下依次调用每个节点的 `backward()` 方法, 自动完成梯度传播。

总结 Backward AD 适用于多输入标量输出的梯度计算, 尤其在神经网络训练中效率极高。其核心优势在于: 相较于 Forward AD 每个方向导数需传播一次, Backward AD 可一次性计算所有输入变量的梯度。PyTorch 通过计算图、函数节点和上下文管理, 实现了灵活而高效的反向自动微分机制。